PYTHON ACTIVITY-BASED LEARNING

10 Selected Case Studies — Diagram Flowcharts Edition

Problem statements, objectives, assumptions, algorithms, pseudocode, Python code, test cases, sample outputs and diagram-style flowcharts.

Student Name:	Sonti Mahitha Sri
Roll Number:	26000199
Section:	CSE-3

Flowcharts use standard diagram shapes: rounded rectangles for Start/Stop, parallelogram-style input/output is represented with labeled boxes, rectangles for processing, and diamonds for decisions.

1. Progressive Electricity Billing

Problem statement: Calculate a household electricity bill using progressive unit slabs and a fixed charge.

Objectives

• Practise variables, arithmetic and if-elif-else.

• Apply progressive slabs to calculate the energy charge.

Assumptions

• First 100 units: Rs. 1.50/unit; next 100: Rs. 2.50/unit; next 100: Rs. 4/unit; above 300: Rs. 6/unit.

• Fixed charge Rs. 50. Rates are illustrative, not official tariffs.

Algorithm

1. Start.

2. Input units consumed.

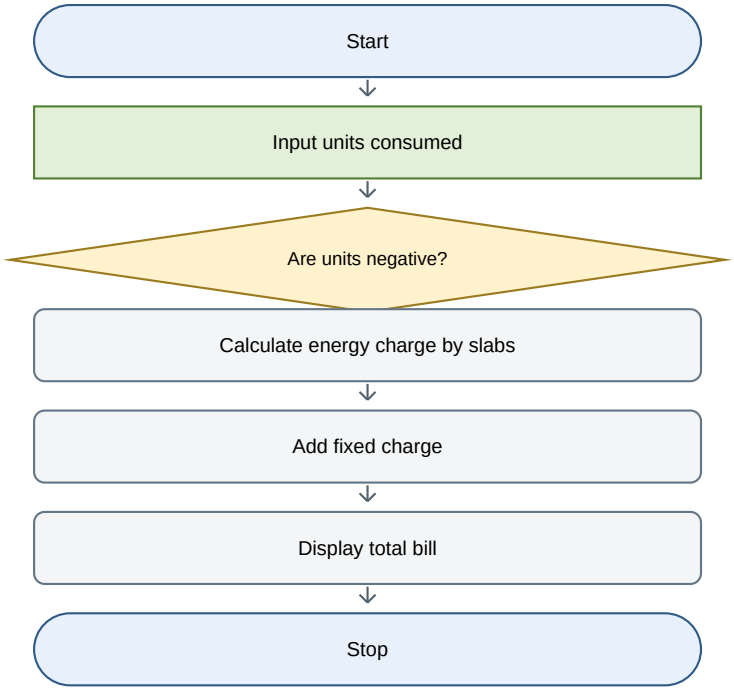
3. Validate that units are not negative.

4. Calculate the energy charge using the relevant slabs.

5. Add fixed charge and display bill.

6. Stop.

Flowchart — Diagram Representation



Pseudocode

```
START
INPUT units
IF units < 0: DISPLAY error
ELSE calculate energy charge using slabs
total = energy + 50
DISPLAY energy and total
STOP
```

Python Source Code

```
units = int(input("Enter electricity units consumed: "))
fixed_charge = 50

if units < 0:
    print("Units cannot be negative.")
elif units <= 100:
    energy_charge = units * 1.50
elif units <= 200:
    energy_charge = 100 * 1.50 + (units - 100) * 2.50
elif units <= 300:
    energy_charge = 100 * 1.50 + 100 * 2.50 + (units - 200) * 4.00
else:
    energy_charge = 100 * 1.50 + 100 * 2.50 + 100 * 4.00 + (units - 300) * 6.00

if units >= 0:
    total_bill = energy_charge + fixed_charge
    print("Units:", units)
    print("Energy charge: Rs.", round(energy_charge, 2))
    print("Fixed charge: Rs.", fixed_charge)
```

```
print("Total bill: Rs.", round(total_bill, 2))
```

Test Cases

Test input	Expected result
150 units	Energy Rs. 275; total Rs. 325
0 units	Energy Rs. 0; total Rs. 50
-5 units	Error: units cannot be negative

Sample Output

```
Enter electricity units consumed: 150
Units: 150
Energy charge: Rs. 275.0
Fixed charge: Rs. 50
Total bill: Rs. 325.0
```

Conclusion

The program calculates a bill using progressive rates and a fixed charge.

2. Employee Payroll and Deduction Analysis

Problem statement: Calculate gross salary and net salary after a percentage deduction.

Objectives

• Practise variables and arithmetic operations.

• Generate a salary statement.

Assumptions

• Deduction is 10% of gross salary for this exercise.

• All salary components must be non-negative; the rate is illustrative.

Algorithm

1. Start.

2. Input basic pay, allowances, overtime and incentive.

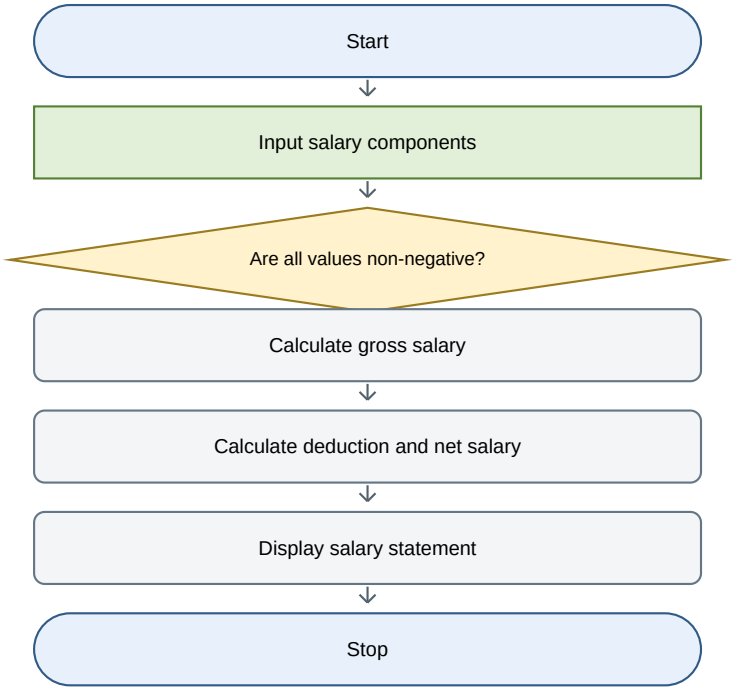
3. Validate all components.

4. Calculate gross salary and 10% deduction.

5. Calculate net salary and display statement.

6. Stop.

Flowchart — Diagram Representation



Pseudocode

```
START
INPUT basic, allowances, overtime, incentive
IF any value < 0: DISPLAY error
ELSE gross = sum of components
deduction = gross * 0.10
net = gross - deduction
DISPLAY gross, deduction, net
STOP
```

Python Source Code

```
basic = float(input("Enter basic salary (Rs.): "))
allowances = float(input("Enter total allowances (Rs.): "))
overtime = float(input("Enter overtime amount (Rs.): "))
incentive = float(input("Enter incentive (Rs.): "))

if min(basic, allowances, overtime, incentive) < 0:
    print("Salary components cannot be negative.")
else:
    gross_salary = basic + allowances + overtime + incentive
    deduction = gross_salary * 0.10
    net_salary = gross_salary - deduction
    print("\n--- Salary Statement ---")
    print("Gross salary: Rs.", round(gross_salary, 2))
    print("Deduction (10%): Rs.", round(deduction, 2))
    print("Net salary: Rs.", round(net_salary, 2))
```

Test Cases

Test input	Expected result
20000, 3000, 1000, 1000	Gross Rs. 25000; net Rs. 22500
10000, 0, 0, 0	Gross Rs. 10000; net Rs. 9000
Negative component	Error message

Sample Output

```
Enter basic salary (Rs.): 20000
Enter total allowances (Rs.): 3000
Enter overtime amount (Rs.): 1000
Enter incentive (Rs.): 1000
Gross salary: Rs. 25000.0
Deduction (10%): Rs. 2500.0
Net salary: Rs. 22500.0
```

Conclusion

The program calculates gross and net salary using the stated assumptions.

3. Railway Fare and Reservation Calculator

Problem statement: Calculate fare using journey distance, travel class, age concession and reservation fee.

Objectives

• Practise conditional statements and arithmetic.

• Apply class rates and age rules.

Assumptions

• Sleeper Rs. 1.20/km; AC Rs. 2.50/km; reservation fee Rs. 40.

• Under age 5 gets full base-fare concession; age 60+ gets 20% off base fare. Illustrative rules only.

Algorithm

1. Start.

2. Input distance, class and age.

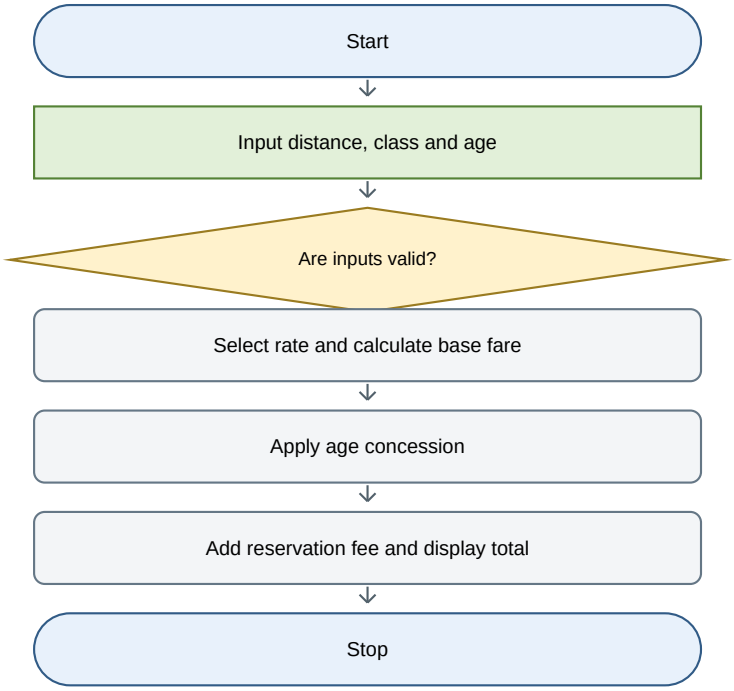
3. Validate inputs and class choice.

4. Calculate base fare and concession.

5. Add reservation fee and display total.

6. Stop.

Flowchart — Diagram Representation



Pseudocode

```
START
INPUT distance, class, age
IF invalid distance or age: DISPLAY error
SELECT rate from class
base = distance * rate
CALCULATE concession from age
total = base - concession + 40
DISPLAY total
STOP
```

Python Source Code

```
distance = float(input("Enter journey distance in km: "))
print("1. Sleeper (Rs. 1.20/km)")
print("2. AC (Rs. 2.50/km)")
choice = int(input("Choose class (1 or 2): "))
age = int(input("Enter passenger age: "))
reservation_fee = 40

if distance < 0 or age < 0:
    print("Distance and age cannot be negative.")
elif choice not in (1, 2):
    print("Invalid class choice.")
else:
    if choice == 1:
        rate, class_name = 1.20, "Sleeper"
    else:
        rate, class_name = 2.50, "AC"
    base_fare = distance * rate
```

```
if age < 5:
    concession = base_fare
elif age >= 60:
    concession = base_fare * 0.20
else:
    concession = 0
total_fare = base_fare - concession + reservation_fee
print("Class:", class_name)
print("Base fare: Rs.", round(base_fare, 2))
print("Concession: Rs.", round(concession, 2))
print("Reservation fee: Rs.", reservation_fee)
print("Total fare: Rs.", round(total_fare, 2))
```

Test Cases

Test input	Expected result
100 km, Sleeper, age 30	Total Rs. 160
100 km, AC, age 65	Total Rs. 240
Negative distance	Error message

Sample Output

```
Enter journey distance in km: 100
1. Sleeper (Rs. 1.20/km)
2. AC (Rs. 2.50/km)
Choose class (1 or 2): 1
Enter passenger age: 30
Class: Sleeper
Base fare: Rs. 120.0
Concession: Rs. 0
Reservation fee: Rs. 40
Total fare: Rs. 160.0
```

Conclusion

The program calculates an illustrative fare using travel class and age-based concession rules.

4. Examination Result and Grace-Mark Validation

Problem statement: Validate marks, apply a limited grace-mark rule, and determine the final result.

Objectives

• Practise lists, loops and conditions.

• Ensure marks stay within 0 to 100.

Assumptions

• Three subjects out of 100; pass mark 35.

• At most one subject may receive up to 3 grace marks; actual policies may differ.

Algorithm

1. Start.

2. Read and validate each subject mark.

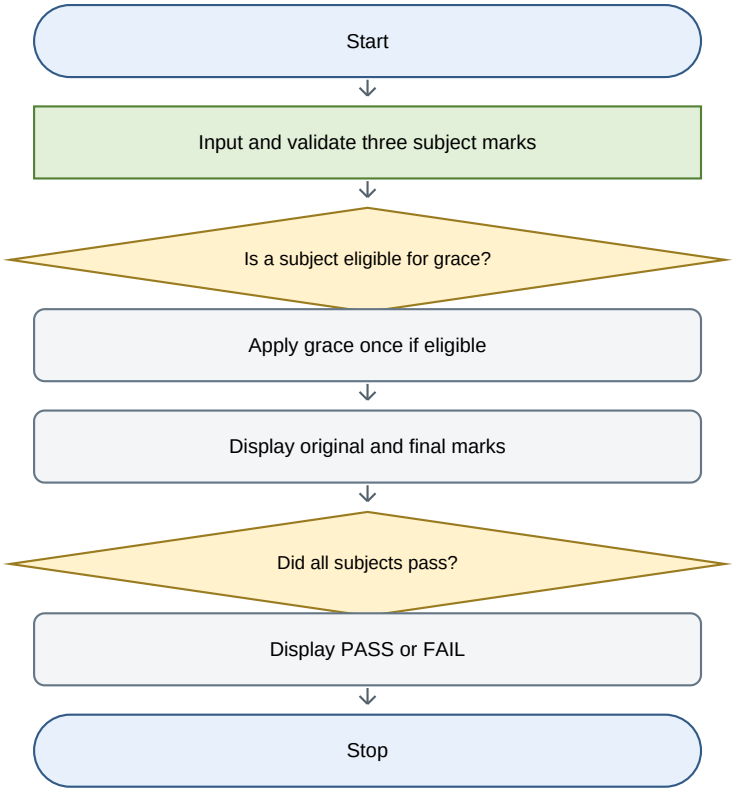
3. Apply grace to at most one eligible subject.

4. Display original and final marks.

5. Display PASS if all final marks are at least 35; otherwise FAIL.

6. Stop.

Flowchart — Diagram Representation



Pseudocode

```
START
INPUT valid marks for three subjects
final_marks = copy of marks
FOR each mark: IF gap to 35 <= 3 and grace unused, add grace
DISPLAY original and final marks
IF all final marks >= 35: PASS ELSE FAIL
STOP
```

Python Source Code

```
subjects = ["English", "Maths", "Python"]
marks = []
pass_mark = 35
grace_limit = 3
grace_used = False

for subject in subjects:
    mark = float(input("Enter marks in " + subject + " (0-100): "))
    while mark < 0 or mark > 100:
        print("Marks must be between 0 and 100.")
        mark = float(input("Re-enter marks in " + subject + ": "))
    marks.append(mark)

final_marks = marks.copy()
```

```
for i in range(len(final_marks)):
    gap = pass_mark - final_marks[i]
    if final_marks[i] < pass_mark and gap <= grace_limit and not grace_used:
        final_marks[i] = min(pass_mark, final_marks[i] + grace_limit)
        grace_used = True

for i in range(len(subjects)):
    print(subjects[i], "Original:", marks[i], "Final:", final_marks[i])

if all(mark >= pass_mark for mark in final_marks):
    print("Result: PASS")
else:
    print("Result: FAIL")
```

Test Cases

Test input	Expected result
35, 33, 70	Maths final 35; PASS
30, 80, 90	FAIL
Invalid 101 then valid marks	Re-prompts for valid mark

Sample Output

```
Enter marks in English (0-100): 35
Enter marks in Maths (0-100): 33
Enter marks in Python (0-100): 70
English Original: 35.0 Final: 35.0
Maths Original: 33.0 Final: 35.0
Python Original: 70.0 Final: 70.0
Result: PASS
```

Conclusion

The program validates marks and applies the specified grace-mark rule before deciding the result.

5. ATM Transaction Sequence Simulation

Problem statement: Simulate deposits, withdrawals and balance inquiries while rejecting invalid transactions.

Objectives

• Practise while loops and conditions.

• Store transactions in tuples and unpack them for a summary.

Assumptions

• Classroom simulation only; no bank fees or overdrafts.

• Amounts must be positive and withdrawals cannot exceed balance.

Algorithm

1. Start and read opening balance.

2. Repeat menu until Exit.

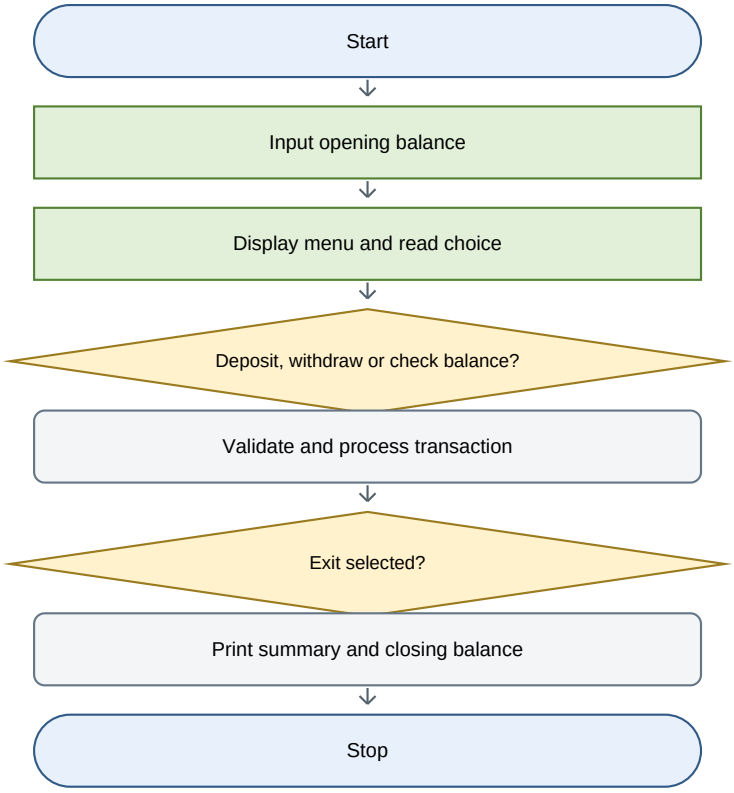
3. Process deposits and valid withdrawals.

4. Display balance when requested.

5. Print accepted transactions and closing balance.

6. Stop.

Flowchart — Diagram Representation



Pseudocode

```
START
INPUT opening balance; initialize transaction list
REPEAT menu until Exit
IF Deposit: validate and add amount
ELSE IF Withdraw: validate amount and balance
ELSE IF Balance: display balance
ELSE IF Exit: stop loop
DISPLAY transaction summary and closing balance
STOP
```

Python Source Code

```
balance = float(input("Enter opening balance (Rs.): "))
transactions = []

while True:
    print("\n1. Deposit")
    print("2. Withdraw")
    print("3. Check balance")
    print("4. Exit")
    choice = input("Choose an option: ")

    if choice == "1":
        amount = float(input("Enter deposit amount: "))
```

```
        if amount > 0:
            balance += amount
            transactions.append(("Deposit", amount))
            print("Deposit successful.")
        else:
            print("Amount must be positive.")
    elif choice == "2":
        amount = float(input("Enter withdrawal amount: "))
        if amount <= 0:
            print("Amount must be positive.")
        elif amount > balance:
            print("Rejected: insufficient balance.")
        else:
            balance -= amount
            transactions.append(("Withdrawal", amount))
            print("Withdrawal successful.")
    elif choice == "3":
        print("Current balance: Rs.", round(balance, 2))
    elif choice == "4":
        break
    else:
        print("Invalid option.")

print("\n--- Transaction Summary ---")
for transaction_type, amount in transactions:
    print(transaction_type, ": Rs.", round(amount, 2))
print("Closing balance: Rs.", round(balance, 2))
```

Test Cases

Test input	Expected result
Open 1000; deposit 500; withdraw 200	Closing balance Rs. 1300
Open 100; withdraw 150	Rejected; balance remains Rs. 100
Deposit -5	Rejected

Sample Output

```
Enter opening balance (Rs.): 1000
Choose an option: 1
Enter deposit amount: 500
Deposit successful.
Choose an option: 2
Enter withdrawal amount: 200
Withdrawal successful.
Choose an option: 4
Deposit : Rs. 500.0
Withdrawal : Rs. 200.0
Closing balance: Rs. 1300.0
```

Conclusion

The program maintains a balance and records only accepted transactions.

6. Warehouse Inventory Reconciliation

Problem statement: Compare expected closing stock with recorded closing stock and identify discrepancies.

Objectives

• Practise lists, indexing and loops.

• Calculate expected stock and report shortages or surpluses.

Assumptions

• Sample data is provided for three products.

• Expected closing stock = opening + received - dispatched.

Algorithm

1. Start.

2. Create product and stock lists.

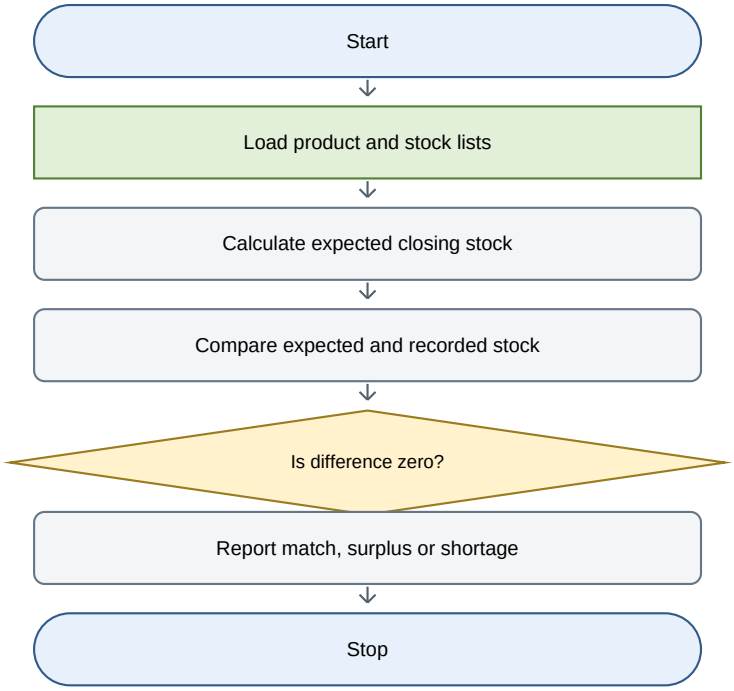
3. For each product, calculate expected closing stock.

4. Compare expected and recorded stock.

5. Report match, shortage or surplus.

6. Stop.

Flowchart — Diagram Representation



Pseudocode

```
START
SET product and stock lists
FOR each product i: expected = opening[i] + received[i] - dispatched[i]
difference = recorded[i] - expected
IF difference = 0: match
ELSE IF difference > 0: surplus
ELSE: shortage
STOP
```

Python Source Code

```
products = ["Notebook", "Pen", "File"]
opening_stock = [100, 200, 50]
received = [30, 50, 20]
dispatched = [40, 80, 10]
recorded_closing = [90, 165, 60]

print("--- Inventory Reconciliation ---")
for i in range(len(products)):
    expected = opening_stock[i] + received[i] - dispatched[i]
    difference = recorded_closing[i] - expected
    print("\nProduct:", products[i])
    print("Expected closing stock:", expected)
    print("Recorded closing stock:", recorded_closing[i])
    if difference == 0:
        print("Status: Stock matches")
    elif difference > 0:
        print("Status: Surplus of", difference)
    else:
```

```
        print("Status: Shortage of", abs(difference))
```

Test Cases

Test input	Expected result
Notebook recorded 90	Expected 90; matches
Pen recorded 165	Expected 170; shortage 5
File recorded 60	Expected 60; matches

Sample Output

```
Product: Notebook
Expected closing stock: 90
Recorded closing stock: 90
Status: Stock matches

Product: Pen
Expected closing stock: 170
Recorded closing stock: 165
Status: Shortage of 5

Product: File
Expected closing stock: 60
Recorded closing stock: 60
Status: Stock matches
```

Conclusion

The program compares inventory records and identifies stock discrepancies.

7. Student Email and Identifier Validation

Problem statement: Check a simple student email-like address and registration ID against sample format rules.

Objectives

• Practise string splitting, slicing and character checks.

• Display valid or invalid status.

Assumptions

• Email validation is basic, not a complete email-standard validator.

• ID format: two digits, three letters, three digits (example: 24CSE001).

Algorithm

1. Start.

2. Input email and student ID.

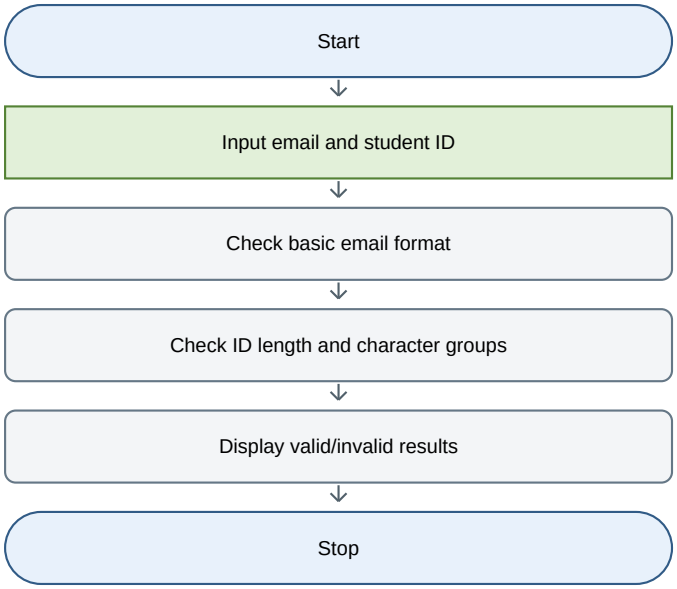
3. Check email has one @, non-empty local part and a dot in the domain.

4. Check ID length and character groups.

5. Display both results.

6. Stop.

Flowchart — Diagram Representation



Pseudocode

```
START
INPUT email, student_id
SPLIT email at @ and check basic conditions
CONVERT student_id to uppercase
CHECK length 8, first 2 digits, next 3 letters, last 3 digits
DISPLAY validation results
STOP
```

Python Source Code

```
email = input("Enter student email: ").strip()
student_id = input("Enter student identifier (e.g. 24CSE001): ").strip().upper()

email_parts = email.split("@")
valid_email = (
    len(email_parts) == 2
    and len(email_parts[0]) > 0
    and "." in email_parts[1]
    and not email_parts[1].startswith(".")
    and not email_parts[1].endswith(".")
)

valid_id = (
    len(student_id) == 8
    and student_id[:2].isdigit()
    and student_id[2:5].isalpha()
    and student_id[5:].isdigit()
)

print("Email:", "Valid" if valid_email else "Invalid")
print("Student ID:", "Valid" if valid_id else "Invalid")
```

Test Cases

Test input	Expected result
student@college.edu; 24CSE001	Both valid
studentcollege.edu; 24CSE001	Email invalid; ID valid
student@college.edu; 2ACSE01	Email valid; ID invalid

Sample Output

```
Enter student email: student@college.edu
Enter student identifier (e.g. 24CSE001): 24CSE001
Email: Valid
Student ID: Valid
```

Conclusion

The program applies simple string-based checks to email and student identifier formats.

8. Student Marks Matrix Analysis

Problem statement: Calculate student totals and averages, subject averages and the top performer using a nested list.

Objectives

• Practise nested lists and nested loops.

• Perform row-wise and column-wise calculations.

Assumptions

• Each row contains marks in the same subject order.

• Marks are out of 100; example data is built into the program.

Algorithm

1. Start.

2. Store student names, subject names and marks matrix.

3. Calculate each student's total and average.

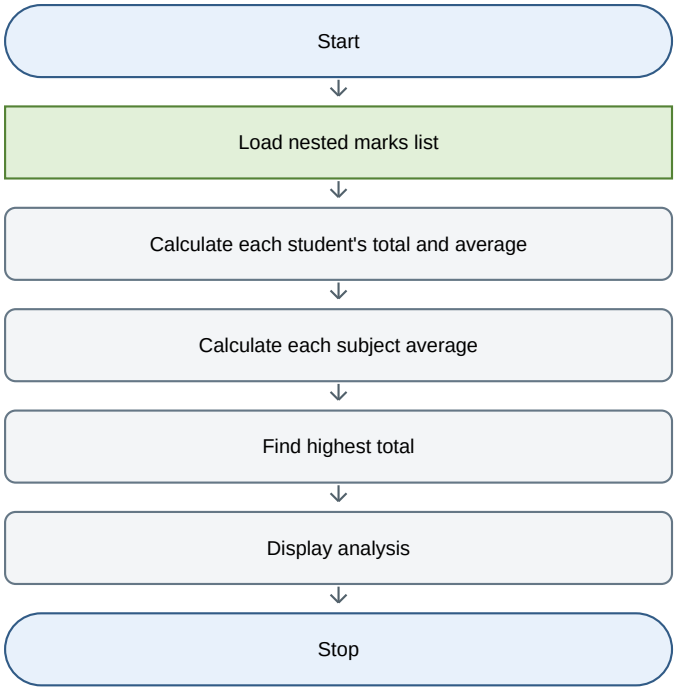
4. Calculate each subject's average.

5. Find the highest total.

6. Display analysis.

7. Stop.

Flowchart — Diagram Representation



Pseudocode

```
START
SET student, subject and marks matrix
FOR each student: calculate total and average
FOR each subject: calculate average across students
FIND student with highest total
DISPLAY results
STOP
```

Python Source Code

```
students = ["Anu", "Bala", "Charan"]
subjects = ["Maths", "Python", "English"]
marks = [
    [85, 90, 78],
    [70, 75, 80],
    [92, 88, 95]
]

print("--- Student Marks Analysis ---")
for i in range(len(students)):
    total = sum(marks[i])
    average = total / len(subjects)
    print(students[i], "Total:", total, "Average:", round(average, 2))

print("\n--- Subject Averages ---")
for j in range(len(subjects)):
    subject_total = 0
```

```
    for i in range(len(students)):
        subject_total += marks[i][j]
    print(subjects[j], "Average:", round(subject_total / len(students), 2))

top_index = 0
for i in range(1, len(students)):
    if sum(marks[i]) > sum(marks[top_index]):
        top_index = i
print("\nTop performer:", students[top_index])
```

Test Cases

Test input	Expected result
Anu: 85,90,78	Total 253; average 84.33
Bala: 70,75,80	Total 225; average 75
Charan: 92,88,95	Total 275; top performer

Sample Output

```
Anu Total: 253 Average: 84.33
Bala Total: 225 Average: 75.0
Charan Total: 275 Average: 91.67

Maths Average: 82.33
Python Average: 84.33
English Average: 84.33

Top performer: Charan
```

Conclusion

The program calculates student and subject statistics from a nested list.

9. Tuple-Based Employee Shift Conflict Detection

Problem statement: Detect overlapping time intervals assigned to the same employee using tuples.

Objectives

• Practise tuples, nested loops and tuple unpacking.

• Compare time intervals to find conflicts.

Assumptions

• Shift tuple format: (employee_name, start_hour, end_hour).

• All shifts are on the same day; end time is excluded.

Algorithm

1. Start.

2. Store shifts as tuples.

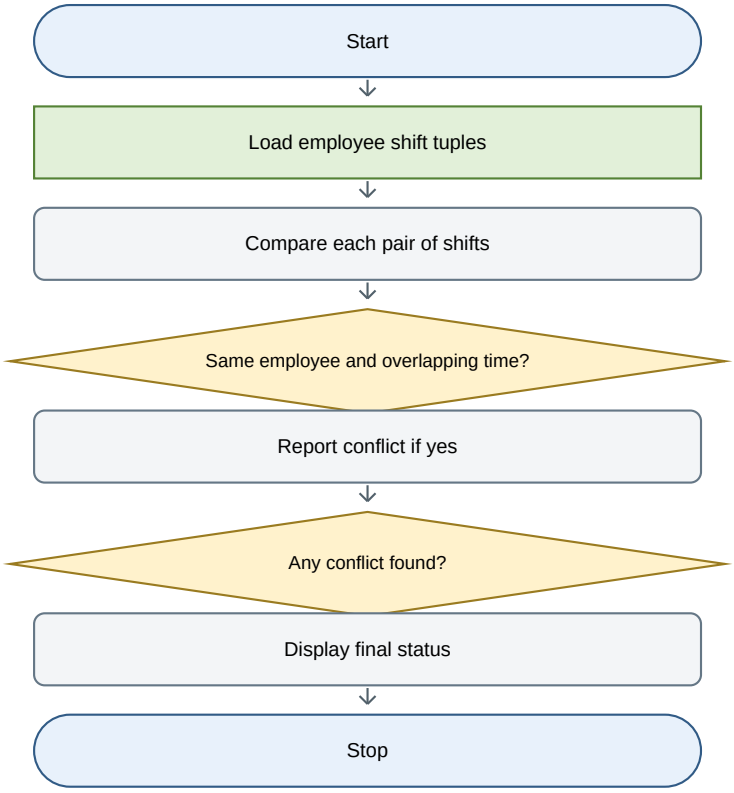
3. Compare each pair of shifts.

4. If names match and intervals overlap, report a conflict.

5. If no conflicts are found, report that none exist.

6. Stop.

Flowchart — Diagram Representation



Pseudocode

```
START
SET shifts list of tuples
FOR each shift i: unpack name and interval
FOR each later shift j: unpack name and interval
IF same name AND start1 < end2 AND start2 < end1: report conflict
IF no conflict: report none
STOP
```

Python Source Code

```
shifts = [
    ("Anu", 9, 13),
    ("Bala", 12, 16),
    ("Charan", 14, 18),
    ("Anu", 11, 12)
]

print("--- Shift Conflict Report ---")
conflict_found = False

for i in range(len(shifts)):
    name1, start1, end1 = shifts[i]
    for j in range(i + 1, len(shifts)):
        name2, start2, end2 = shifts[j]
```

```
        if name1 == name2 and start1 < end2 and start2 < end1:
            print(name1, "has overlapping shifts:",
                  start1, "-", end1, "and", start2, "-", end2)
            conflict_found = True

if not conflict_found:
    print("No employee shift conflicts found.")
```

Test Cases

Test input	Expected result
Anu 9-13 and Anu 11-12	Conflict reported
Anu 9-10 and Anu 10-12	No overlap
Anu 9-11 and Bala 10-12	No same-employee conflict

Sample Output

```
--- Shift Conflict Report ---
Anu has overlapping shifts: 9 - 13 and 11 - 12
```

Conclusion

The program detects overlapping shifts for the same employee using tuple data.

10. Retail Billing with Multiple Discount Rules

Problem statement: Create an itemized bill, apply a threshold discount and calculate tax.

Objectives

• Practise lists, loops and conditional expressions.

• Calculate subtotal, discount, tax and final amount.

Assumptions

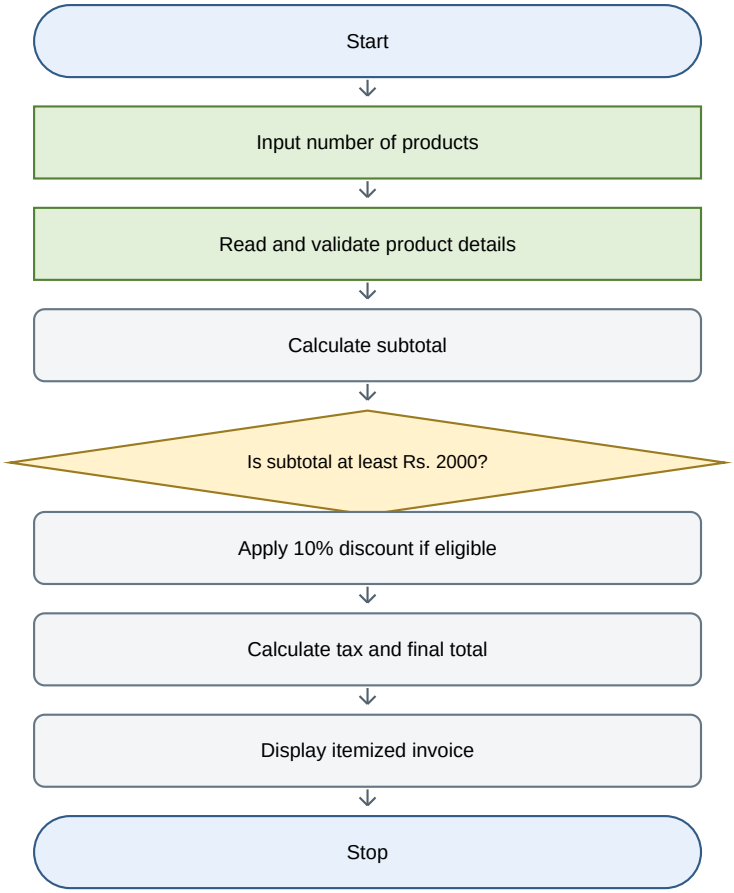
• 10% discount for subtotal of Rs. 2000 or more.

• Tax is 5% after discount; these are sample academic rates.

Algorithm

1. Start.
2. Input number of products and product details.
3. Validate quantity and price.
4. Calculate line amounts and subtotal.
5. Apply discount if threshold is met.
6. Calculate tax and final total.
7. Display invoice.
8. Stop.

Flowchart — Diagram Representation



Pseudocode

```
START
INPUT number of products
FOR each product: INPUT name, quantity, price; validate and store
subtotal = sum(quantity * price)
IF subtotal >= 2000: discount = 10% ELSE discount = 0
tax = (subtotal - discount) * 5%
total = subtotal - discount + tax
DISPLAY invoice
STOP
```

Python Source Code

```
products = []
quantities = []
prices = []
```

```
n = int(input("Enter number of products: "))
if n <= 0:
    print("Number of products must be positive.")
else:
    for i in range(n):
        print("\nProduct", i + 1)
        name = input("Enter product name: ")
        quantity = int(input("Enter quantity: "))
        price = float(input("Enter price per item (Rs.): "))
        while quantity < 0 or price < 0:
            print("Quantity and price cannot be negative. Re-enter details.")
            quantity = int(input("Enter quantity: "))
            price = float(input("Enter price per item (Rs.): "))
        products.append(name)
        quantities.append(quantity)
        prices.append(price)

    subtotal = 0
    print("\n--- Itemized Invoice ---")
    for i in range(n):
        amount = quantities[i] * prices[i]
        subtotal += amount
        print(products[i], "x", quantities[i], "=", "Rs.", round(amount, 2))

    discount = subtotal * 0.10 if subtotal >= 2000 else 0
    taxable_amount = subtotal - discount
    tax = taxable_amount * 0.05
    total = taxable_amount + tax
    print("Subtotal: Rs.", round(subtotal, 2))
    print("Discount: Rs.", round(discount, 2))
    print("Tax (5%): Rs.", round(tax, 2))
    print("Final total: Rs.", round(total, 2))
```

Test Cases

Test input	Expected result
2 items at Rs. 1000 each	Subtotal 2000; discount 200; tax 90; total 1890
1 item at Rs. 500	No discount; tax 25; total 525
Quantity -1	Prompts for valid details

Sample Output

```
Enter number of products: 2
Product 1: Bag, quantity 1, price Rs. 1000
Product 2: Book, quantity 1, price Rs. 1000
Subtotal: Rs. 2000.0
Discount: Rs. 200.0
Tax (5%): Rs. 90.0
Final total: Rs. 1890.0
```

Conclusion

The program produces an itemized bill and calculates discount, tax and final total.